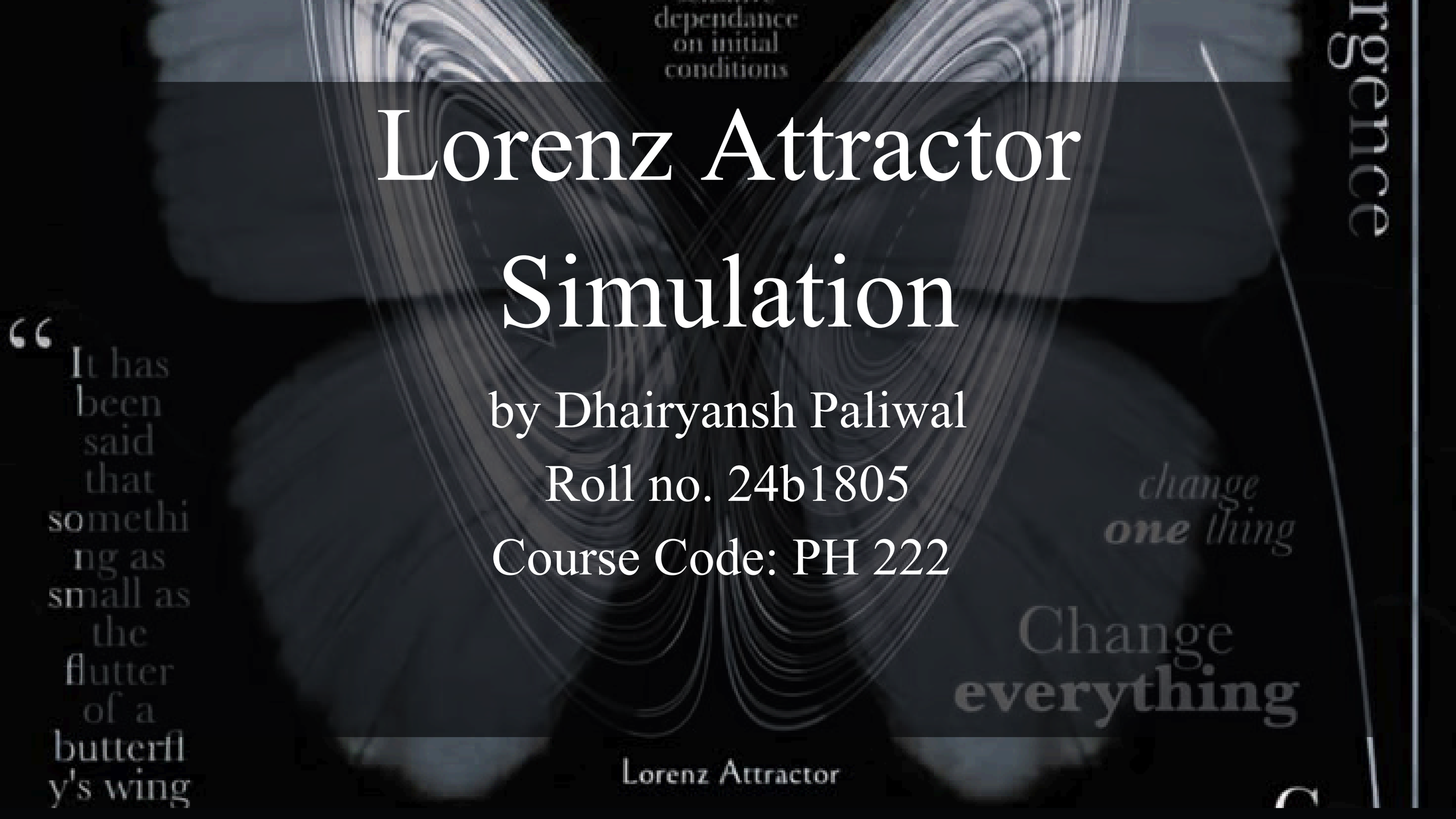


Lorenz Attractor Simulation

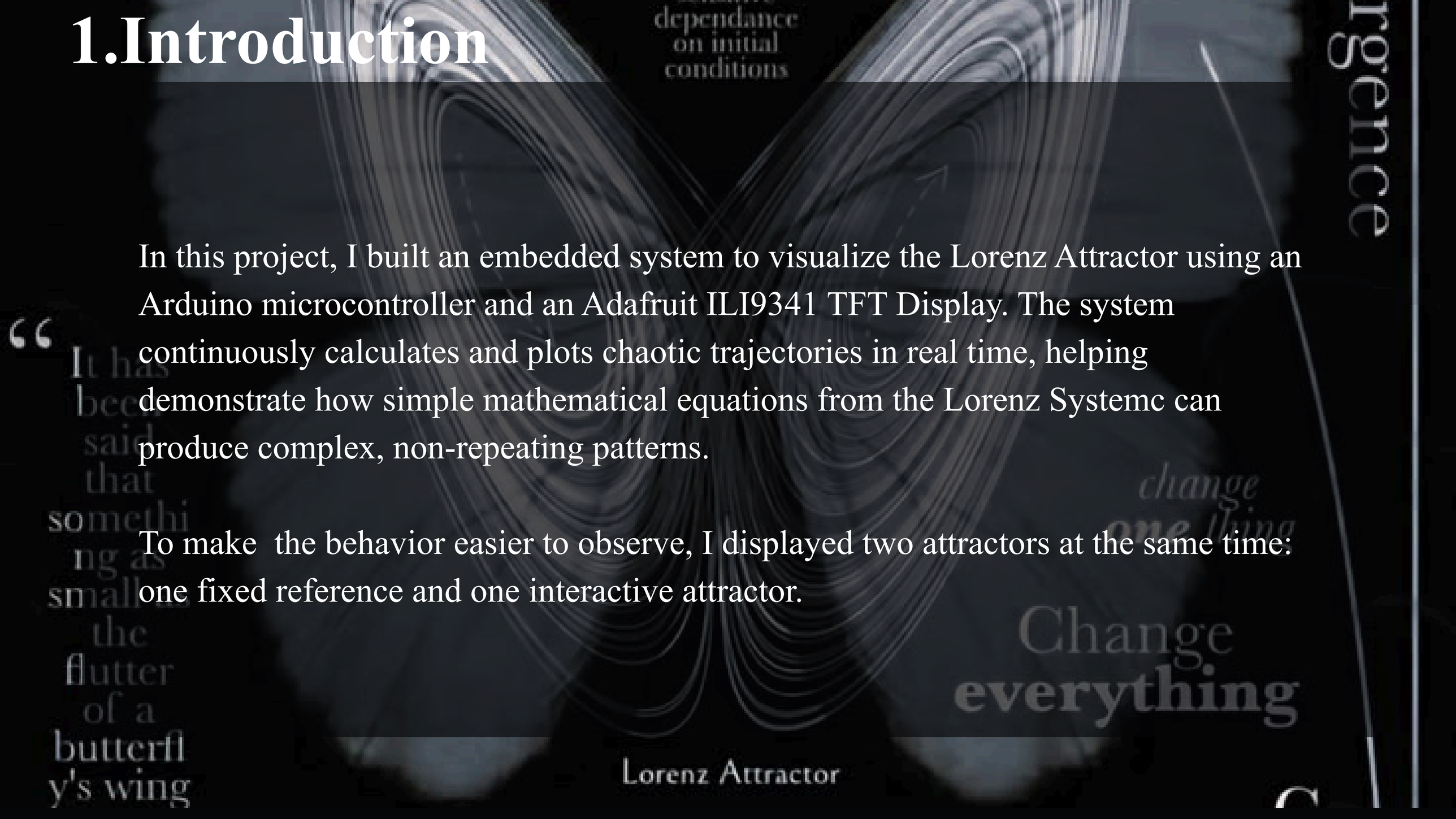
by Dhairyansh Paliwal
Roll no. 24b1805

Course Code: PH 222

1.Introduction

In this project, I built an embedded system to visualize the Lorenz Attractor using an Arduino microcontroller and an Adafruit ILI9341 TFT Display. The system continuously calculates and plots chaotic trajectories in real time, helping demonstrate how simple mathematical equations from the Lorenz Systemc can produce complex, non-repeating patterns.

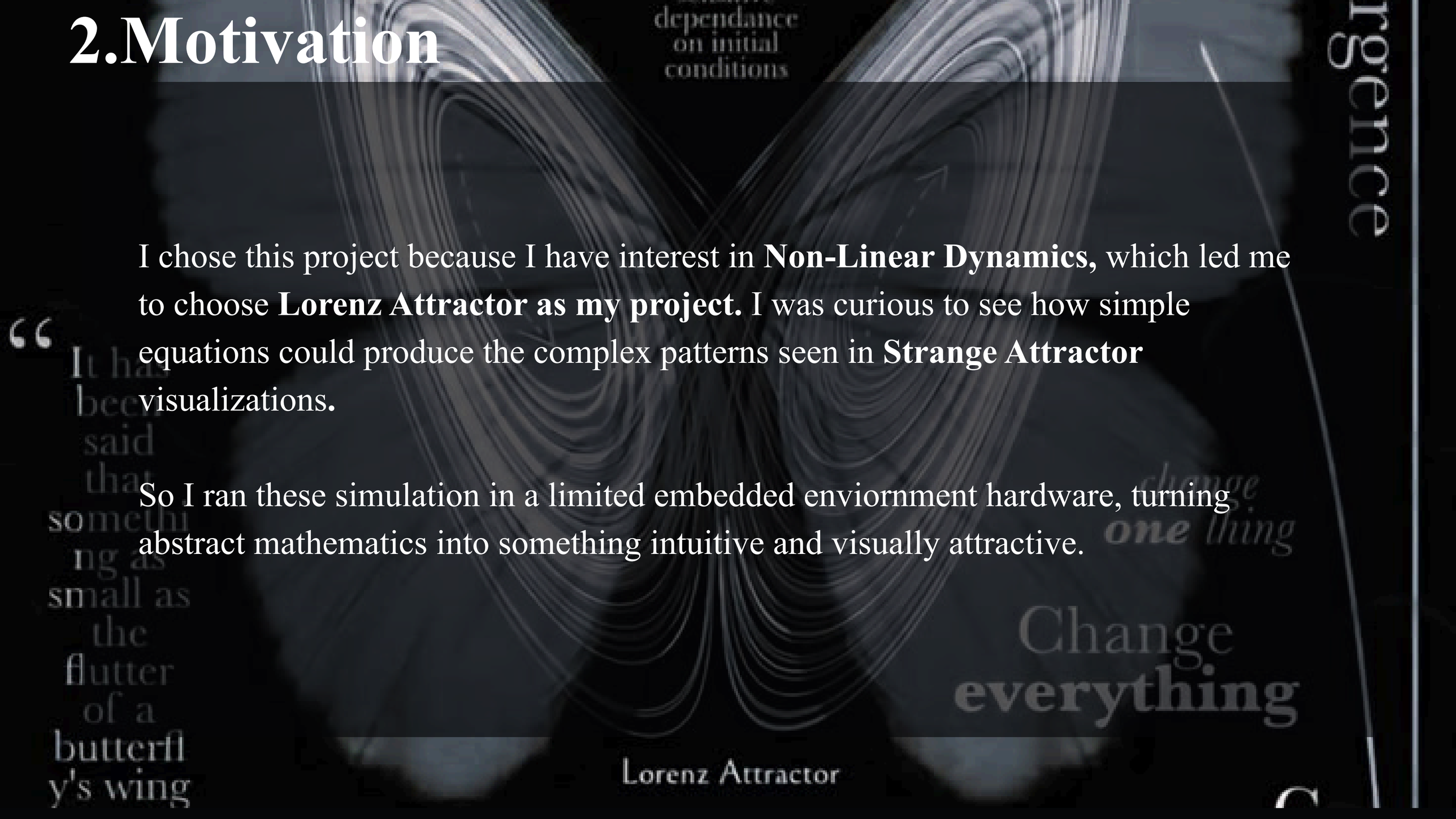
To make the behavior easier to observe, I displayed two attractors at the same time: one fixed reference and one interactive attractor.



2.Motivation

I chose this project because I have interest in **Non-Linear Dynamics,** which led me to choose **Lorenz Attractor as my project.** I was curious to see how simple equations could produce the complex patterns seen in **Strange Attractor** visualizations**.**

So I ran these simulation in a limited embedded enviornment hardware, turning abstract mathematics into something intuitive and visually attractive.

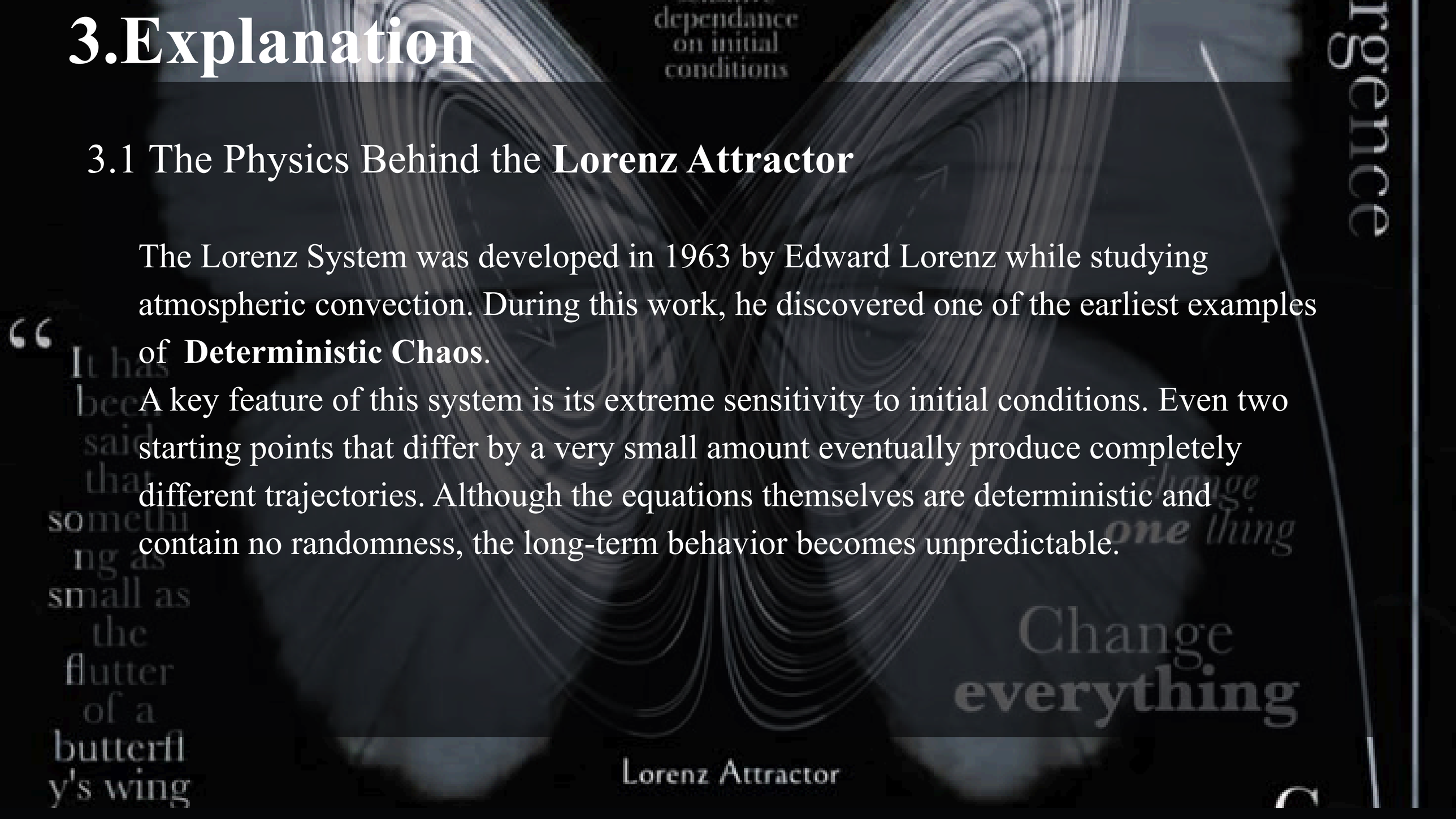


3.Explanation

3.1 The Physics Behind the **Lorenz Attractor**

The Lorenz System was developed in 1963 by Edward Lorenz while studying atmospheric convection. During this work, he discovered one of the earliest examples of **Deterministic Chaos**.

A key feature of this system is its extreme sensitivity to initial conditions. Even two starting points that differ by a very small amount eventually produce completely different trajectories. Although the equations themselves are deterministic and contain no randomness, the long-term behavior becomes unpredictable.



3.Explanation

3.2 The Lorenz Equations

The motion of the system is described by three coupled, non-linear differential equations:

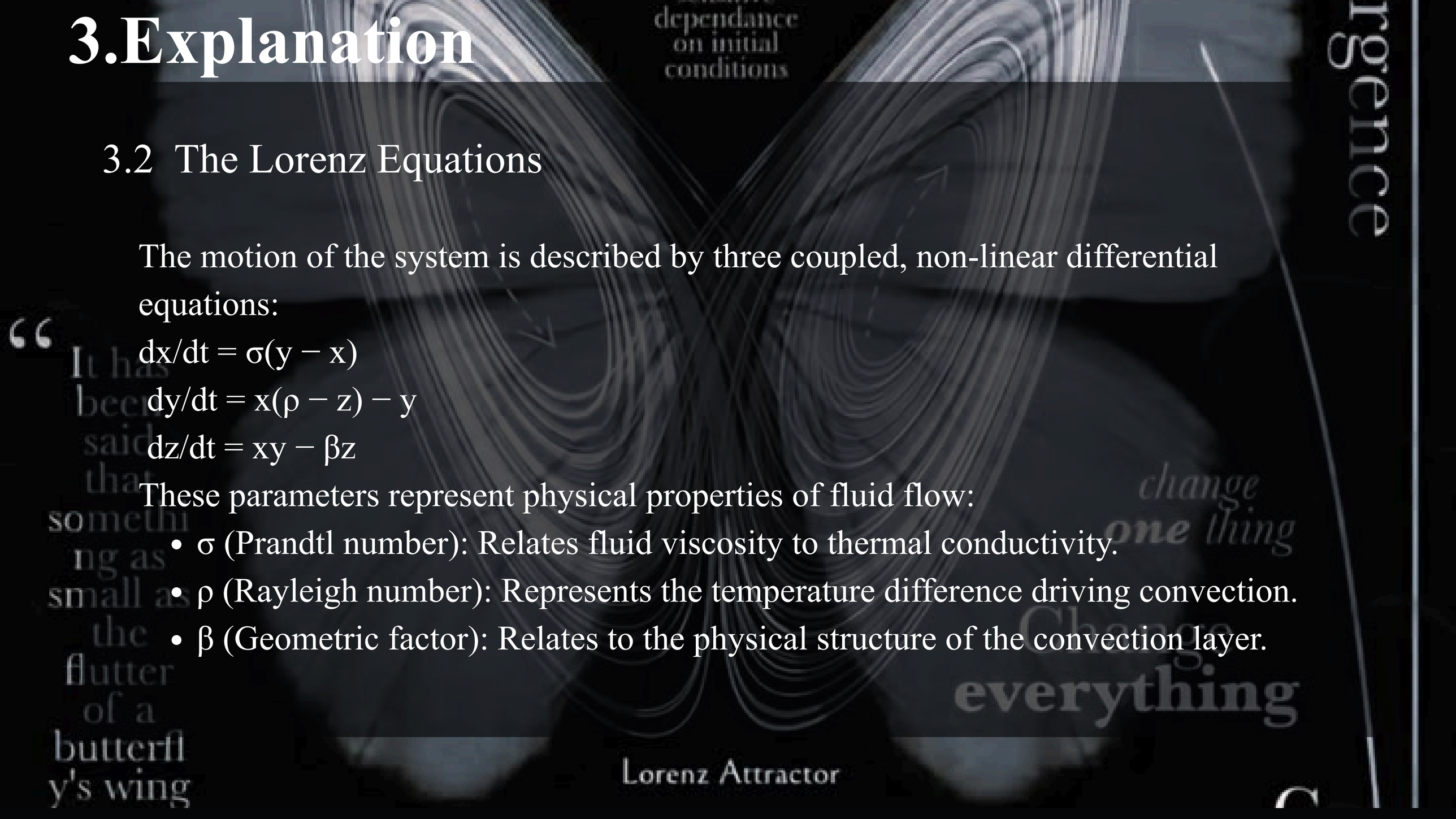
$dx/dt = \sigma(y - x)$

$dy/dt = x(\rho - z) - y$

$dz/dt = xy - \beta z$

These parameters represent physical properties of fluid flow:

- σ (Prandtl number): Relates fluid viscosity to thermal conductivity.
- ρ (Rayleigh number): Represents the temperature difference driving convection.
- β (Geometric factor): Relates to the physical structure of the convection layer.

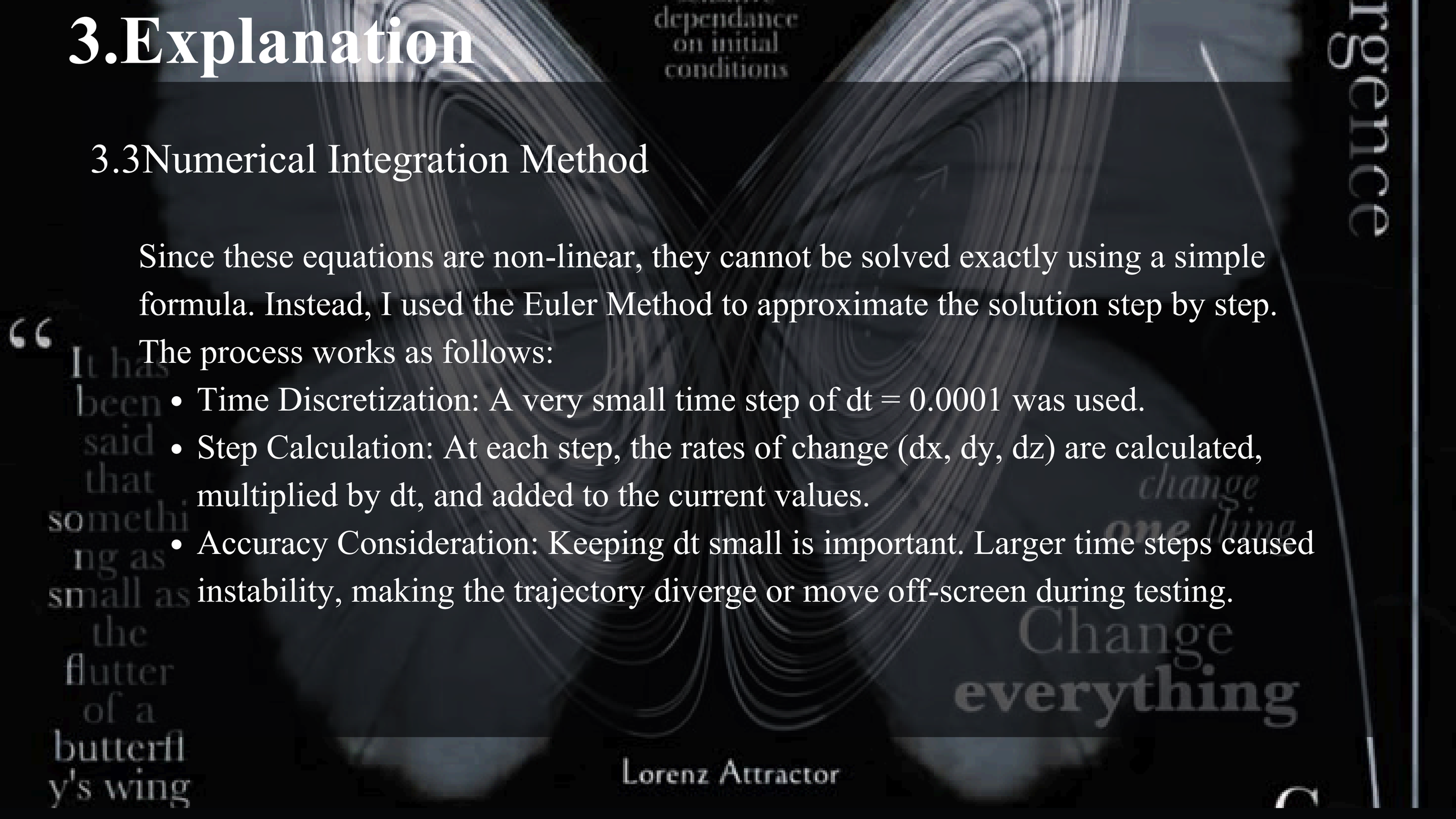


3.Explanation

3.3Numerical Integration Method

Since these equations are non-linear, they cannot be solved exactly using a simple formula. Instead, I used the Euler Method to approximate the solution step by step. The process works as follows:

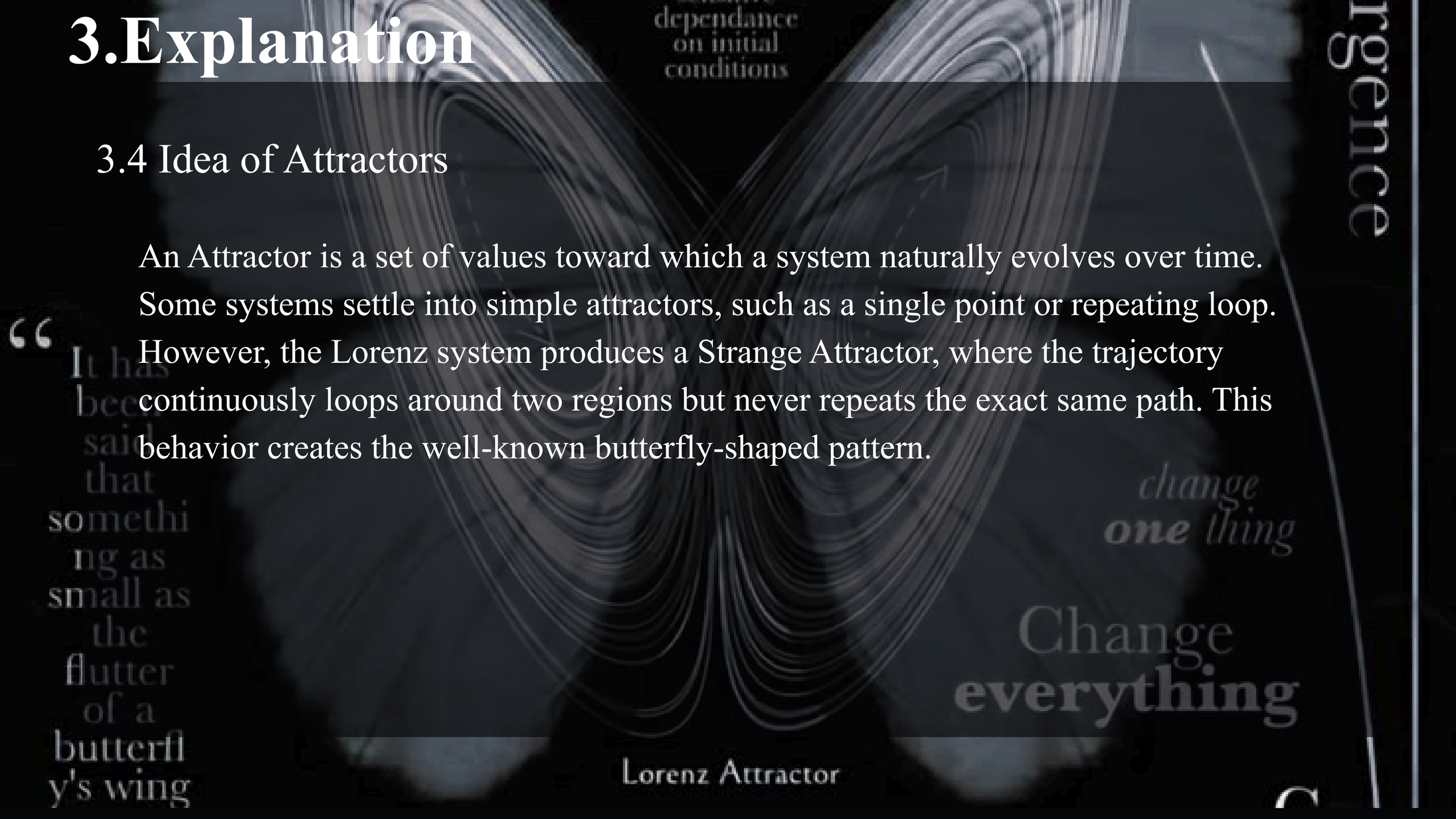
- Time Discretization: A very small time step of dt = 0.0001 was used.
- Step Calculation: At each step, the rates of change (dx, dy, dz) are calculated, multiplied by dt, and added to the current values.
- Accuracy Consideration: Keeping dt small is important. Larger time steps caused instability, making the trajectory diverge or move off-screen during testing.



3.Explanation

3.4 Idea of Attractors

An Attractor is a set of values toward which a system naturally evolves over time. Some systems settle into simple attractors, such as a single point or repeating loop. However, the Lorenz system produces a Strange Attractor, where the trajectory continuously loops around two regions but never repeats the exact same path. This behavior creates the well-known butterfly-shaped pattern.

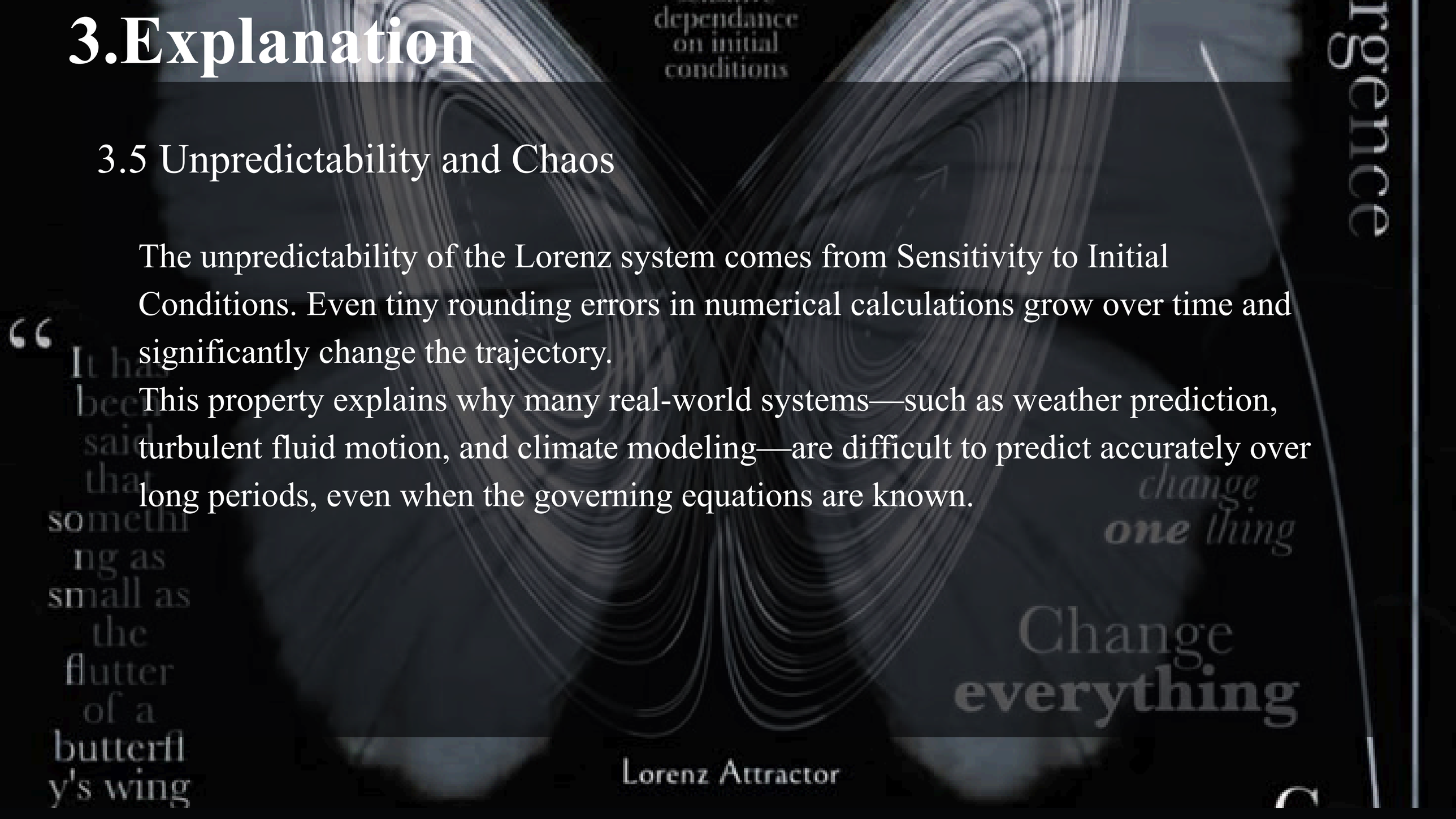


3.Explanation

3.5 Unpredictability and Chaos

The unpredictability of the Lorenz system comes from Sensitivity to Initial Conditions. Even tiny rounding errors in numerical calculations grow over time and significantly change the trajectory.

This property explains why many real-world systems—such as weather prediction, turbulent fluid motion, and climate modeling—are difficult to predict accurately over long periods, even when the governing equations are known.



4.Process of building

4.1 Simulation over Wokwi and the Challenges

Before soldering physical components, I simulated the architecture using Wokwi.

- Testing TFT Behavior: I needed to ensure the Adafruit ILI9341 library worked correctly.
- Coordinates and Scaling: A major challenge was that the mathematical variables range roughly from -20 to 20, but the screen pixels are 320x240. I had to implement a mapping function: map(x * 20, -400, 400, 0, 320) to scale the simulation up and center it on the display.
- Challenges: The Wokwi simulation revealed heavy display flickering if I cleared the screen every frame. I solved this by only drawing new pixels, completely avoiding tft.fillScreen() during the main drawing loop.

4.Process of building

4.2 Building the Circuit and the Challenges

Moving to physical hardware presented tangible problems.

- Wiring: I wired the TFT display using the hardware SPI bus, assigning pins 10 (CS), 8 (DC), and 9 (RST). The rotary encoder was connected to pins 2, 3, and 4.
- Issues Faced: Initially, I encountered a completely white screen. By double-checking my wiring, I realized my SPI clock and data lines were swapped. Furthermore, the rotary encoder was floating, giving random inputs, which I fixed by declaring the pins as INPUT_PULLUP.

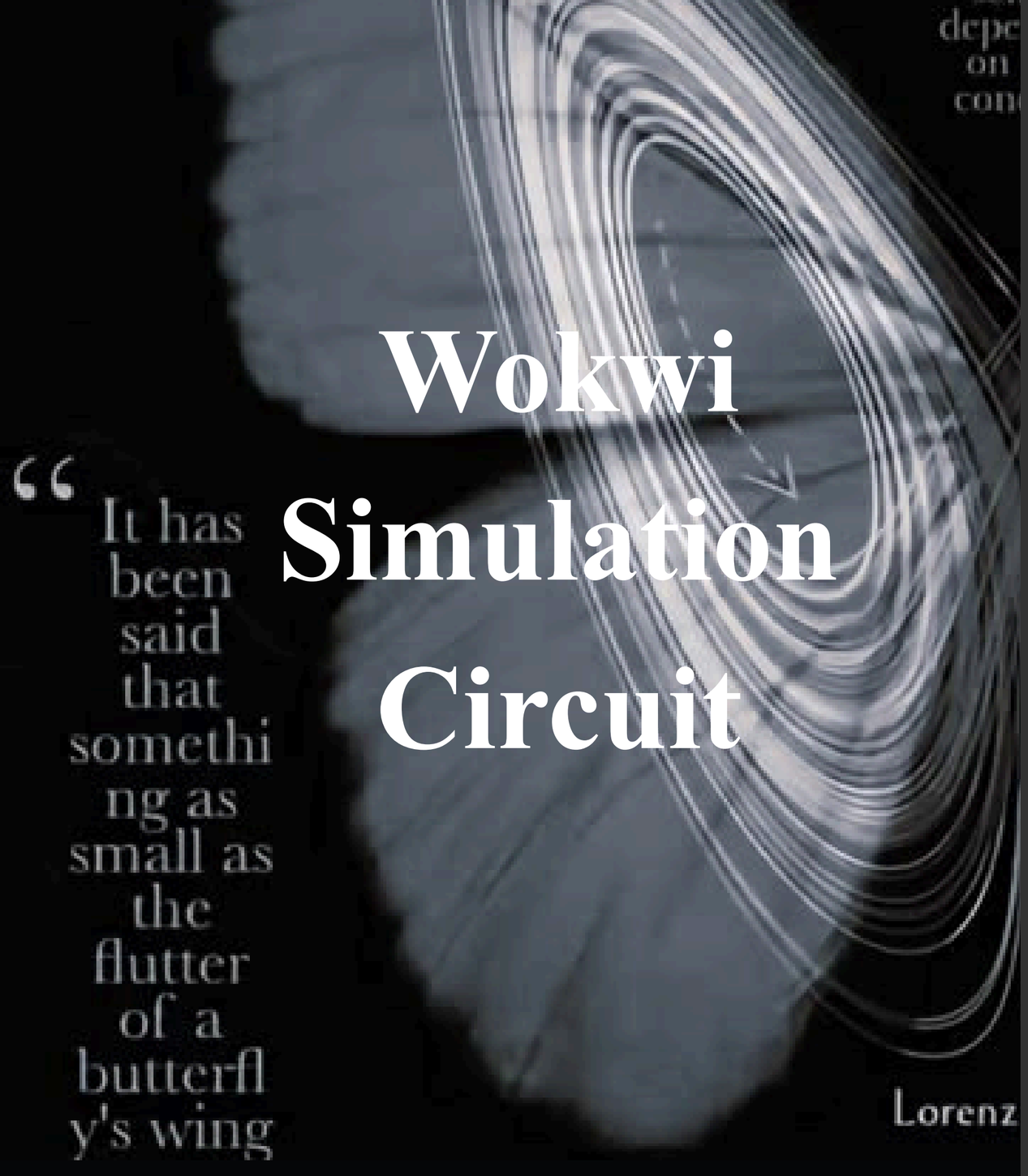
4.Process of building

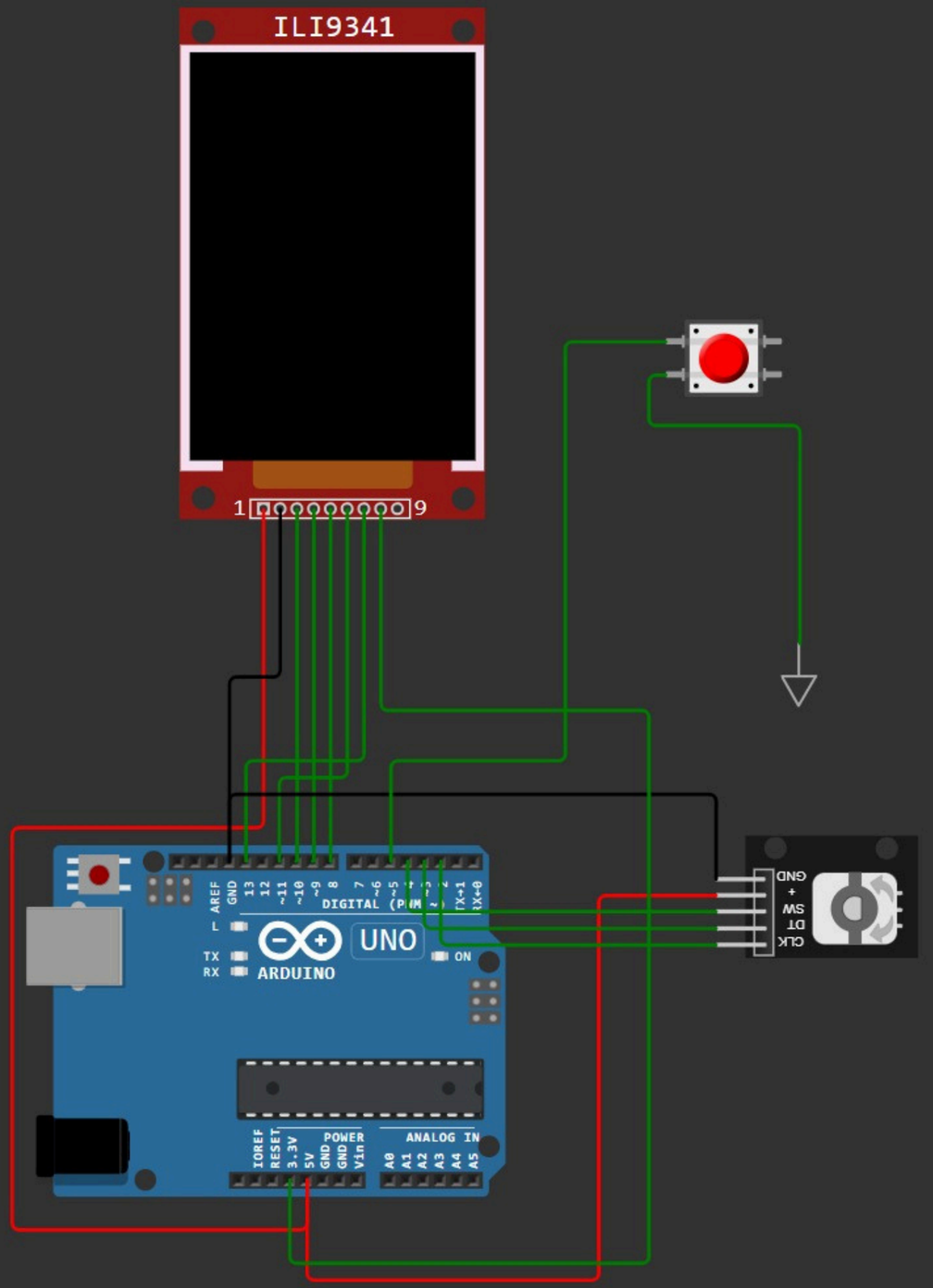
4.3 Coding Errors and How I Overcame Them

Translating the math into C++ was a significant learning curve.

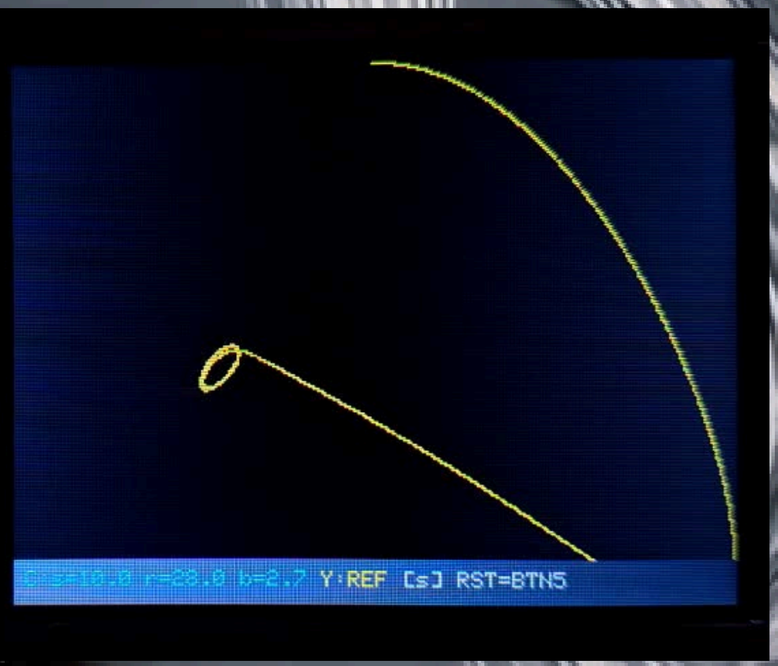
- Encoder Logic: Reading physical rotation is tricky. I initially used basic digital reads, but it missed steps. I had to research and implement bitwise shifting (sum == 0b1101...) inside a hardware interrupt to properly track rotational direction.
- Debouncing: The button to reset the parameters back to their defaults (10, 28, 2.67) was triggering multiple times from one press. I implemented a software debounce using a millis() timer check, enforcing a 200ms delay between reads.
- Parameter Boundaries: I realized that spinning the encoder too far caused the variables to crash the math. I added constrain() functions to lock σ between 0 and 50, ρ between 0 and 100, and β between 0 and 10.

Wokwi Simulation Circuit





Simulations



same initial conditions
and parameters

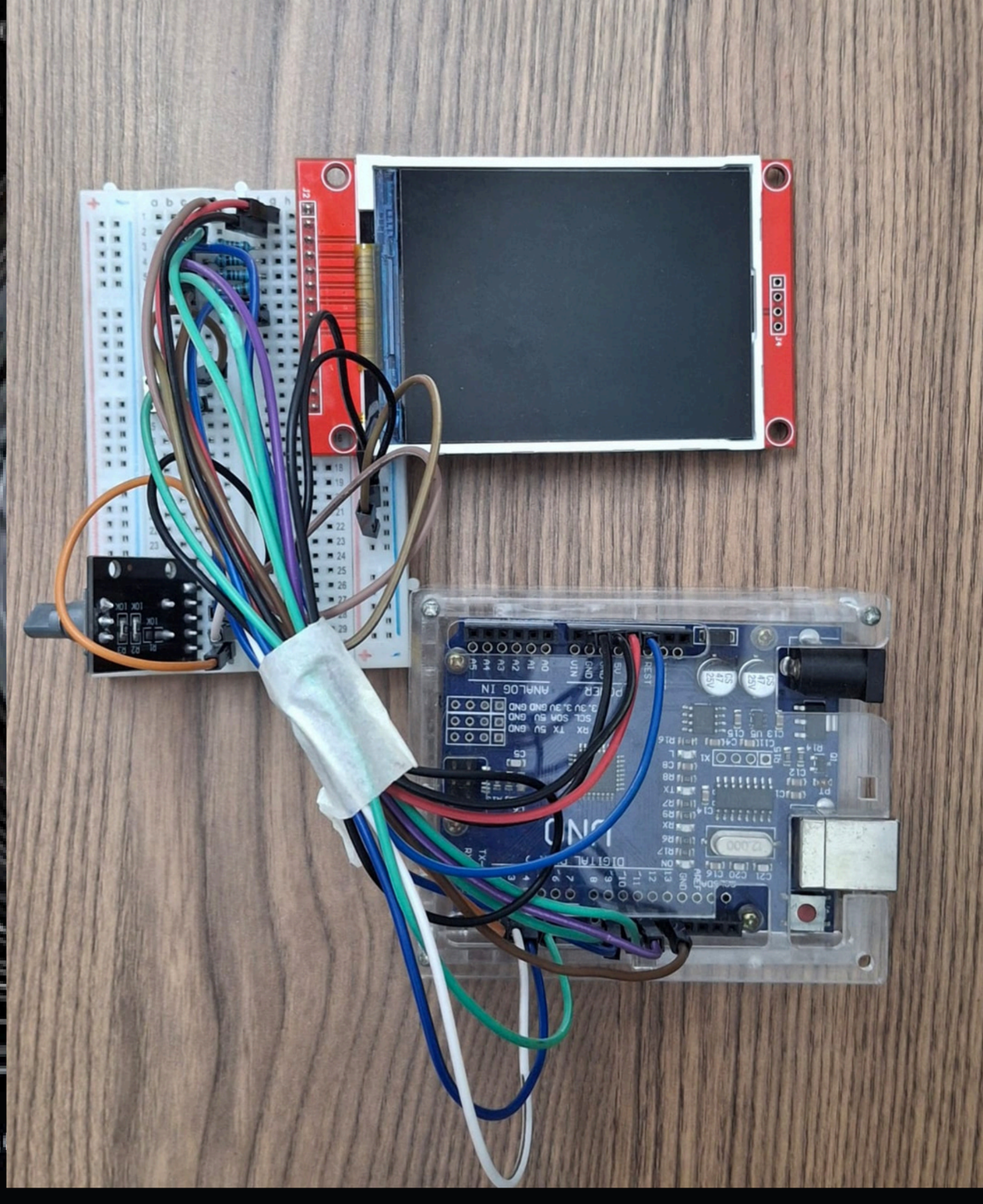


same initial conditions
but different parameters

Lorenz Attractor

Final Setup


dependance
on initial
conditions
“ It has been said that something as small as the flutter of a butterfly's wing
Lorenz Attra



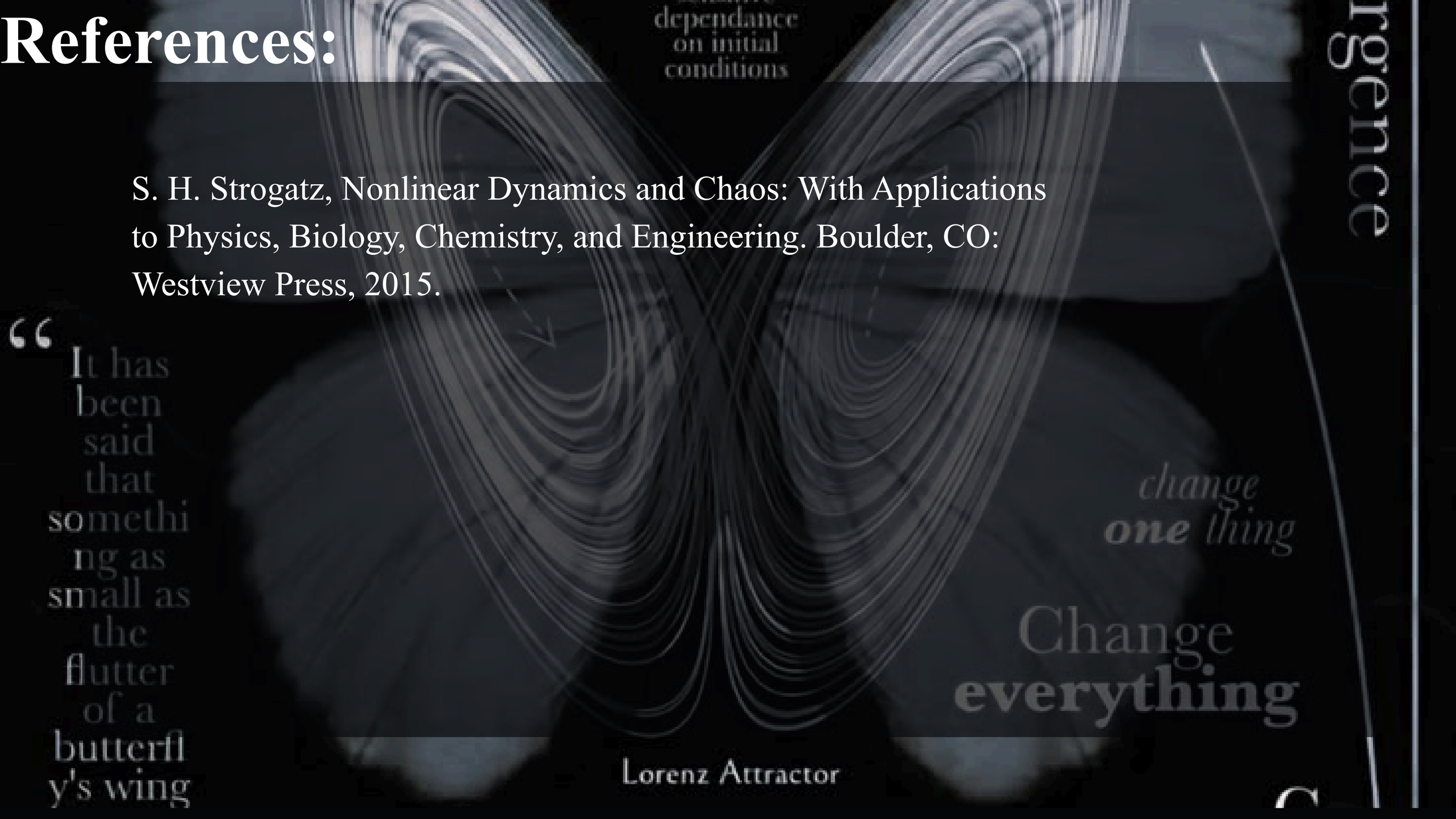
5. **Result**

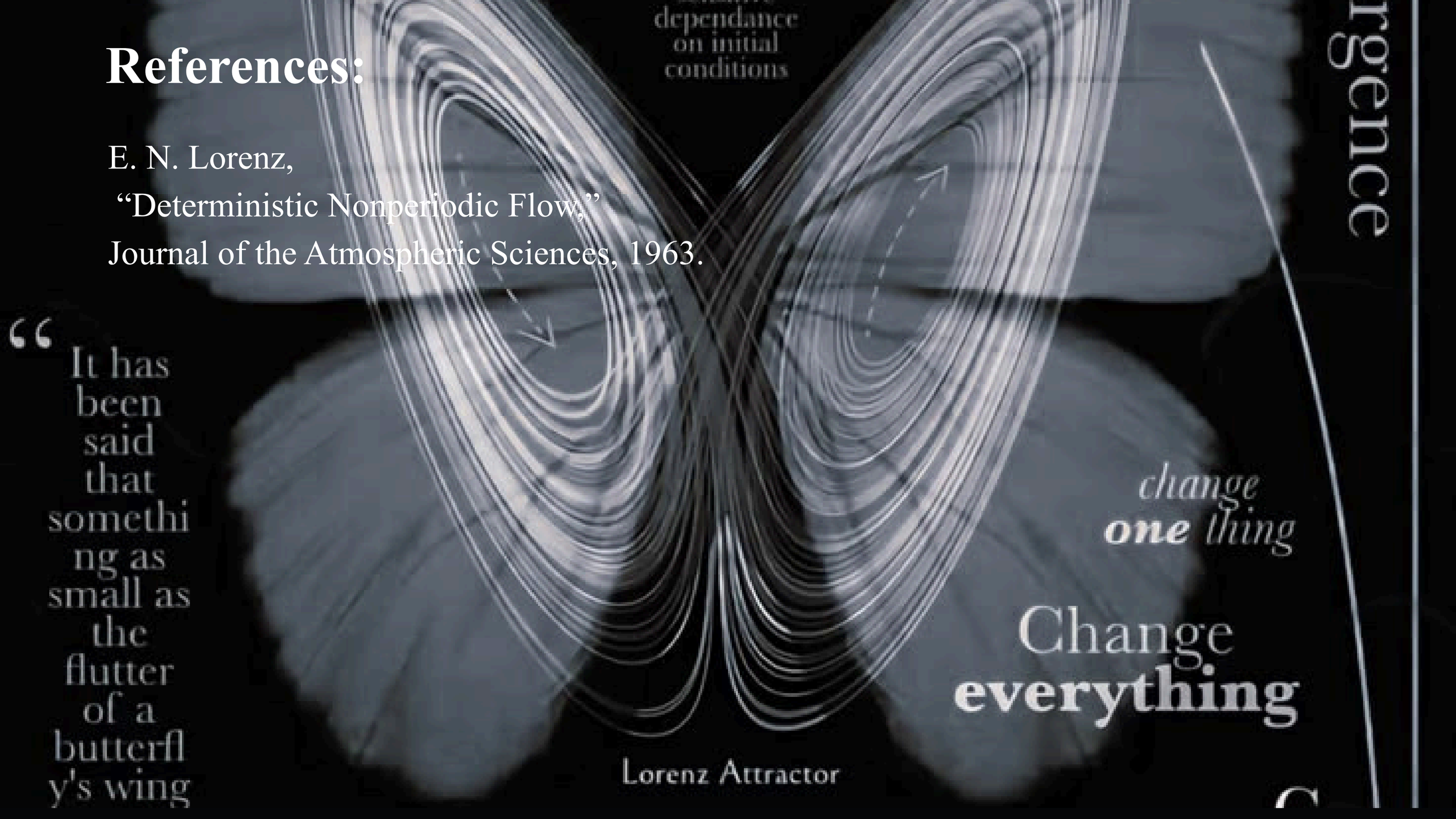
The final output is a stable and interactive visualizer that renders two Lorenz Attractor trajectories simultaneously. The yellow path serves as a reference, while the cyan path updates in real time as parameters are adjusted.

A small UI panel displays the selected variable and its value. Using the Euler Method, the system plots hundreds of points per second while staying within Arduino memory limits. Even slight changes to σ, ρ, or β quickly cause the cyan path to diverge from the reference, clearly showing the system's sensitivity to small changes.

References:

S. H. Strogatz, Nonlinear Dynamics and Chaos: With Applications to Physics, Biology, Chemistry, and Engineering. Boulder, CO: Westview Press, 2015.





References:

E. N. Lorenz,
“Deterministic Nonperiodic Flow,”
Journal of the Atmospheric Sciences, 1963.